

REDROB HACKATHON

Backend Logic Bible — Revised Edition

Semantic-First · Soft Penalties · LLM Re-Ranking

Complete module-by-module logic. No code. No UI. Pure implementation decisions.

0. Revised Principles — What Changed and Why

The original blueprint was solid engineering but over-indexed on hard-coded rules. The four improvements requested, and how this document addresses each one:

Feedback Point	Old Approach	New Approach in This Document
Keep overall architecture	All 6 components + behavioral multiplier + honeypot detection	Retained. Pipeline structure, feature groups, and output format unchanged.
Reduce hard-coded penalties	Binary cliff-edges: all-consulting = 0.1 multiplier; skill duration 0 = trust 0	Smooth sigmoid and power curves everywhere. Minimum floors prevent total zeroing. Penalties are proportional, not binary.
Add LLM re-ranking for final candidates	Template-based ranking only; no LLM in pipeline	Two-phase pipeline: algorithmic first-pass produces top 200-300, then Gemini/GPT-4o-mini re-ranks them into final top 100.
Let semantic understanding drive ranking	30% semantic, 70% keyword trust	Flipped: 60% semantic (multi-query embeddings), 40% keyword trust. Semantic is now the primary signal.

CORE PHILOSOPHY SHIFT: A TCS engineer who built a production recommendation engine for 3 years is better than a 'product company' junior who wrote API wrappers. The new system discovers this through semantic evidence in descriptions and LLM judgment — not through company-name rules.

1. Complete Pipeline Overview — 10 Stages

The system runs in two distinct phases: OFFLINE (pre-computed once) and ONLINE (runs within the 5-minute constraint per ranking job).

Stage	Phase	Module	Input	Output	Feeds Frontend
1	OFFLINE	JD Parser	job_description.docx	requirements.json: skill groups, weights, disqualifier rules, implicit signals, JD embeddings (3 queries)	S3 JD Analyzer
2	OFFLINE	Embedding Engine	candidates.jsonl (all 100K)	candidate_embeddings.npy: shape (100K, 384), one vector per candidate	S8 Score Breakdown, S11
3	OFFLINE	Feature Pre-computer	candidates.jsonl	features_cache.pkl: all non-embedding features for 100K candidates	S8, S9 Analytics
4	ONLINE	Data Loader	candidates.jsonl or uploaded .json/.jsonl	Stream of normalized candidate dicts	All screens
5	ONLINE	Honeypot Detector	Each candidate dict	Per-candidate honeypot_confidence: 0.0 (clean) → 1.0 (impossible)	S8 Honeypot Panel
6	ONLINE	First-Pass Scorer	Normalized candidates + embeddings + features_cache	raw_score per candidate (0-1), full feature vector	S9 Analytics
7	ONLINE	Candidate Shortlister	All raw_scores	Top 200-300 candidates for LLM re-ranking	S11 Re-Ranking Inspector
8	ONLINE	LLM Re-Ranker	Top 200-300 candidate summaries + JD context	llm_score (0-100) + llm_reason per candidate	S8 LLM Panel, S11
9	ONLINE	Blend & Final Rank	raw_score + llm_score per candidate	blended_score, final rank 1-100, final reasoning	S7 Results, S8
10	ONLINE	Output Writer	Final ranked 100	submission.csv (candidate_id, rank, score, reasoning)	S10 Export

TIME BUDGET: Stages 1-3 run OFFLINE (once, not in the 5-minute window). Stages 4-10 run ONLINE and must complete in <5 minutes. With pre-computation: stages 4-10 take approximately 90-120 seconds on CPU for the full 100K dataset.

2. Module 1 — JD Parser

2.1 Purpose and Output Contract

The JD Parser runs once at startup (or offline). It reads `job_description.docx` and produces a structured `requirements` object that every other module consumes. The parser's job is not just extraction — it is interpretation. It must understand what the role truly needs, not just list the words used.

2.2 Extraction Logic — 5 Layers

Layer 1: Hard Skill Groups with Semantic Coverage

For each required skill group, extract both the explicit keywords AND build a semantic coverage field that captures plain-language equivalents. This is critical for the semantic-first approach.

Skill Group	Primary Keywords	Semantic Equivalents to Include
Embeddings & Dense Retrieval	sentence-transformers, SBERT, OpenAI Ada, BGE, E5, embedding drift	built semantic search, representation learning, dense vector matching, finding similar items, similarity search, recommendation by content
Vector Databases & ANN Search	FAISS, Pinecone, Qdrant, Weaviate, Milvus, Elasticsearch ANN, OpenSearch	approximate nearest neighbor, vector store, scalable search infrastructure, similarity lookup at scale, indexed vector retrieval
Retrieval & Ranking Systems	BM25, hybrid search, re-ranking, learning to rank, LambdaMART, NDCG, MRR	ranking pipeline, relevance scoring, search quality, candidate scoring system, matching engine, relevance model
LLMs & NLP	fine-tuning, LoRA, QLoRA, PEFT, RLHF, NLP, transformers, BERT, RAG	language understanding, generative AI, model training, text classification, question answering, document understanding
Evaluation & Experimentation	NDCG, MAP, MRR, A/B testing, offline evaluation, experiment design	measuring search quality, ranking metrics, experiment framework, model performance tracking, online vs offline correlation
Production ML Engineering	FastAPI, model serving, latency, throughput, inference, production systems	deployed to production, served real users, scaled the system, handled traffic, built APIs, monitoring and alerting

Layer 2: Three JD Embedding Queries (Multi-Query Approach)

Instead of embedding the entire JD as one vector (which averages everything and loses specificity), create three targeted embedding queries. These three vectors will be used in the Semantic Embedding Engine.

- **Query 1 — Core Technical Fit:** 'Senior AI engineer building large-scale retrieval and ranking systems with embeddings, vector search, information retrieval, hybrid search, and evaluation frameworks for machine learning models in production.'
- **Query 2 — Production & Engineering Mindset:** 'Production deployment, shipping models to real users at scale, performance optimization, latency, monitoring, owning systems end to end, working in fast-paced startup environment with small team.'

- **Query 3 — Domain Context:** 'HR technology, talent intelligence, job matching, candidate ranking, recruiter tools, AI for hiring, people analytics, talent acquisition platform.'

These three queries capture different dimensions of candidate fit that a single JD embedding would blur together.

Layer 3: Disqualifier Rules (Converted to Soft Scoring Functions)

Instead of binary disqualifiers, convert each to a scoring function that outputs a penalty multiplier in range [0.15, 1.0]. No disqualifier fully zeros a score — the LLM re-ranker may override a borderline case.

Disqualifier Rule	Detection Logic	Penalty Function Output
All-consulting career	Compute $\text{consulting_ratio} = (\text{months at consulting firms}) / (\text{total career months})$	$\text{penalty} = 1.0 - (\text{consulting_ratio}^{0.65} \times 0.75)$. At 100% consulting: 0.25. At 50%: 0.64. At 0%: 1.0.
Pure research, no production	Scan all career descriptions for production_keywords. $\text{production_evidence_score} = \text{count of production keywords across all descriptions, weighted by recency}$	$\text{penalty} = 0.20 + 0.80 \times \min(1.0, \text{production_evidence_score} / 5)$. At 0 evidence: 0.20. At 5+ signals: 1.0.
Recent LangChain-only AI experience	Count months of pre-LLM AI experience (before 2022) vs total AI experience	$\text{penalty} = 0.35 + 0.65 \times \min(1.0, \text{pre_llm_ai_months} / 18)$. At 0 pre-LLM: 0.35. At 18+ months: 1.0.
Non-coding technical leadership (>18m)	Check if current title contains lead/architect/VP AND descriptions lack coding evidence in last 18 months	$\text{penalty} = 0.60 + 0.40 \times \text{coding_evidence_score}$. At 0 coding evidence: 0.60. At full evidence: 1.0.
Computer Vision/Speech only — no NLP/IR	Check if dominant skills are CV/Speech/Robotics with zero NLP/IR/search skills present	$\text{penalty} = 0.40 + 0.60 \times \max(0, \text{nlp_ir_skill_score})$. Smooth: some NLP = partial credit.

Layer 4: Location Scoring Map

Location Bucket	Score	Rationale
Pune or Noida	1.00	JD explicitly names these — no commute, immediate onsite possible
Delhi NCR (Gurgaon, Faridabad, Noida adjacent)	0.92	Same metro as Noida — practical relocation distance
Hyderabad, Mumbai, Bangalore	0.85	Tier-1 tech cities, reasonable relocation
Chennai, Ahmedabad, Kolkata, Jaipur	0.75	Tier-1/2 cities, willing candidates will relocate
Other India locations	0.60	Further from target cities but same country
Outside India + willing_to_relocate = true	0.35	Possible but visa/logistics overhead
Outside India + willing_to_relocate = false	0.15	Impractical — heavy penalty

Layer 5: Experience Scoring Curve

A smooth bell-curve centered at 7 years (midpoint of 5-9 ideal range). Beyond 15 years, scores drop more steeply — this captures the concern about non-coding technical leads.

```
| experience_score = exp(-0.5 × ((YoE - 7) / 3.5) ^ 2)
```

This gives: YoE=7 → 1.0, YoE=5 → 0.83, YoE=9 → 0.83, YoE=3 → 0.53, YoE=12 → 0.60, YoE=15 → 0.35

3. Module 2 — Data Streamer & Normalizer

3.1 Streaming Strategy

The full `candidates.jsonl` is 465 MB. Loading it entirely into RAM uses ~2.5 GB. The correct approach streams one line at a time, processes it, stores only the (`candidate_id`, `raw_score`, `feature_vector`) in memory, and discards the full candidate dict.

- Stream line by line using a buffered file reader. Parse each line with `json.loads`.
- After scoring, store a lightweight result tuple: (`candidate_id`, `raw_score`, `key_features_dict`) where `key_features_dict` contains only the 6 component scores + behavioral multiplier + honeypot_confidence.
- Peak RAM budget: `candidate_embeddings.npy` (~400MB) + `features_cache.pkl` (~100MB) + result tuples (~50MB) = ~550MB total. Comfortably under 16GB constraint.

3.2 Field Normalization Table

Field	Raw Problem	Normalization Rule
location	'Bengaluru, Karnataka', 'Bangalore', 'BLR'	Lowercase, strip. Map variants: bengaluru→bangalore, blr→bangalore, ncr→delhi ncr, gurugram→gurgaon
skills[].name	'BERT', 'bert', 'BERT-base', 'bert model'	Lowercase, strip punctuation. Apply alias map: 'bert'→'transformers', 'gpt'→'llm', 'chatgpt'→'llm', 'faiss'→'faiss' (keep), 'elasticsearch'→'elasticsearch' (keep)
career_history[].company	'TCS', 'Tata Consultancy Services Limited', 'tcs ltd'	Lowercase, strip legal suffixes (ltd, limited, inc, pvt). Compare against consulting firm list case-insensitively.
years_of_experience	Float or integer, sometimes negative/impossible	Clamp to [0, 45]. If negative: treat as 0. If > 45: treat as 45 (data error, not honeypot by itself).
redrob_signals.last_active_date	ISO date string	Convert to <code>days_since_active</code> = <code>(processing_date - parse(last_active_date)).days</code> . Store as integer.
redrob_signals.offer_acceptance_rate	-1 means no prior offer history	Map -1 → None. In scoring, treat None as neutral (0.50) — no history neither helps nor hurts.
redrob_signals.github_activity_score	-1 means no GitHub account	Map -1 → None. Treat None as neutral (0.0 bonus, 0.0 penalty).
career_history[].description	Varies 0 to 2000+ characters	Keep raw. Concatenate all descriptions per candidate into single <code>candidate_description_text</code> for embedding and keyword scanning.

3.3 Candidate Text Construction for Embedding

Each candidate needs one unified text string for semantic embedding. Build it in priority order — most important signals first, since embedding models may truncate long inputs:

```
candidate_text = headline + '. ' + summary + '. '
+ CONCAT(title + ' at ' + company + ': ' + description for each
career role, latest first)
+ 'Skills: ' + COMMA_JOIN(top 12 skills by proficiency)
```

Truncate at 512 tokens (the model's context window). Latest career roles appear first so truncation removes the oldest history, not the most recent.

4. Module 3 — Honeypot Detector (Soft Scoring)

4.1 Philosophy Change

The blueprint used binary flags: one impossible data point = full zero. The revised approach uses confidence accumulation: each rule violation adds evidence. Only when evidence accumulates past a threshold does the penalty become severe. This prevents false positives from legitimate but unusual profiles.

4.2 Four Check Categories — Evidence Points

Check	What It Detects	Evidence Points Added	Notes
Timeline Gap Check	duration_months in career_history differs from (end_date - start_date) / 30 by more than 8 months	30 points per violation	Tolerance of 8 months handles part-time, overlapping projects, rounding
Impossible Date Check	career role start_date before plausible working age (calculated from YoE + current date)	40 points per role violation	Plausible start: current_year - YoE - 25 at earliest
Overlapping Roles Check	Two career entries with overlapping date ranges of more than 3 months	25 points per pair	3-month overlap tolerance handles transition periods
Expert Skill Zero Duration	proficiency = 'expert' AND duration_months = 0 AND endorsements = 0 AND skill NOT in assessment_scores	35 points per skill	This is the keyword-stuffer's giveaway
Advanced Skill Zero Duration	proficiency = 'advanced' AND duration_months = 0	15 points per skill	Less severe than expert — could be recently acquired
Impossible Education	end_year < start_year OR degree duration < 1 year	30 points per violation	Hard minimum — no legitimate 6-month bachelor's degree
Experience-Career Mismatch	years_of_experience claim vs sum(career_history.duration_months) / 12 differs by more than 4 years	25 points total	4-year tolerance accounts for gaps, freelancing not in history
Expert Overload	claims 'expert' in 9+ different skill categories simultaneously	20 points total	Soft flag — possible but suspicious

4.3 Honeypot Confidence Score

Accumulate all evidence points across all violations, then convert to confidence score 0-1:

```
raw_evidence = sum of all evidence points from all violations
honeypot_confidence = 1 - exp(-raw_evidence / 60)
```

This sigmoid-like formula means: 0 points → 0.0 confidence (clean), 60 points → 0.63, 120 points → 0.86, 200+ points → near 1.0.

4.4 Penalty Application

Honeypot Confidence	Penalty Applied to Final Score	Reasoning
0.0 – 0.30	No penalty	Clean or minor anomalies within tolerance
0.30 – 0.55	Multiply base score by 0.55	Multiple soft flags — reduce but don't eliminate
0.55 – 0.75	Multiply base score by 0.15	Strong evidence of impossible profile
0.75 – 1.00	Set final score = 0.0	Disqualified — impossible profile beyond doubt

LLM OVERRIDE RULE: If the LLM re-ranker gives a candidate a score > 70/100 AND the honeypot_confidence is < 0.60, restore the candidate's eligibility. The LLM may recognize legitimate but unusual career patterns that the rule-based detector flags incorrectly. If honeypot_confidence > 0.75: no override — this is too strong.

5. Module 4 — Semantic Embedding Engine

5.1 Multi-Query Embedding Strategy

The core shift from the blueprint: instead of comparing a single JD embedding against a single candidate embedding, use the three targeted JD queries from Module 1 (Section 2.2). This gives three separate similarity scores that capture different dimensions of fit.

5.2 Embedding Model Choice

Model	Dimensions	Speed (CPU, 100K docs)	Quality	Recommendation
all-MiniLM-L6-v2	384	~25 min	Good general purpose	RECOMMENDED — best speed/quality balance for this task
all-mpnet-base-v2	768	~55 min	Better quality	Use if you have time; better semantic discrimination
BAAI/bge-small-en-v1.5	384	~22 min	Retrieval-tuned	Good alternative — specifically optimized for retrieval tasks
OpenAI text-embedding-ada-002	1536	API calls only	Best quality	Offline pre-computation only — banned during online ranking

Pre-compute all candidate embeddings ONCE and store as .npy file. At ranking time, load the file (seconds) and compute cosine similarities in vectorized numpy operations (~5 seconds for 100K). Never re-embed during online ranking.

5.3 Three-Score Computation

For each candidate, compute three similarity scores using the three JD query embeddings:

```
sim_technical = cosine_similarity(jd_query_1_embedding,
candidate_embedding)
sim_production = cosine_similarity(jd_query_2_embedding,
candidate_embedding)
sim_domain = cosine_similarity(jd_query_3_embedding,
candidate_embedding)
```

5.4 Composite Semantic Score

Combine the three scores with weights that reflect the JD's emphasis:

```
semantic_score = 0.60 × sim_technical + 0.30 × sim_production + 0.10 ×
sim_domain
```

Cosine similarity returns values in [-1, 1]. For professional text, values typically fall in [0.1, 0.8]. Normalize by (value - 0.1) / 0.7 to stretch the practical range to [0, 1].

5.5 What Semantic Scoring Catches That Keywords Miss

Plain Language Description	Keywords Present	Semantic Score	Why It Works
'Built a system to match job seekers with relevant postings based on text similarity'	None of: FAISS, embeddings, vector search, NLP	High (0.65+)	'Text similarity matching for job seekers' is semantically close to the JD's retrieval/ranking system description
'Designed relevance scoring model for e-commerce search; reduced irrelevant results by 40%'	Maybe: 'search'	High (0.70+)	'Relevance scoring for search' maps directly to the core JD requirement
'Led a team building recommendation engine that served 10M users daily'	Maybe: 'recommendation'	Medium-High (0.60+)	Recommendation at scale captures production + retrieval dimensions
'Marketing automation specialist, increased click-through rates using A/B tests'	'A/B testing' might keyword-match	Low (0.25)	The overall context is marketing, not AI/ML engineering — semantic correctly penalizes this

6. Module 5 — Skill Scorer (Semantic-First Hybrid)

6.1 The Hybrid Approach

The final skill score blends two signals: keyword-trust (explicit skill claims with evidence validation) and semantic similarity (from Module 4). The semantic score now carries 60% weight — it is the primary signal.

```
| skill_score = 0.40 × keyword_trust_score + 0.60 × semantic_score
```

6.2 Keyword Trust Score — Revised with Soft Curves

The keyword trust score measures how credible the candidate's skill claims are. The revision replaces cliff-edge zeros with smooth curves using soft floors.

Step 1: Proficiency Base Weight

Proficiency Level	Base Weight	Reasoning
Expert	1.00	Full credit — maximum claim
Advanced	0.80	Strong claim, still significant
Intermediate	0.55	Competent, not deep expertise
Beginner	0.25	Acknowledged but not a strength

Step 2: Duration Factor (Sigmoid Curve, No Cliff-Edge)

Replace the old linear scale (duration/24) with a sigmoid function that gives partial credit for short durations:

```
| duration_factor = 0.20 + 0.80 × (1 - e^(-duration_months / 15))
```

This gives: 0 months → 0.20 (minimum floor, not zero), 12 months → 0.61, 24 months → 0.80, 36+ months → 0.93+

The 0.20 floor is intentional: a skill listed on a profile with zero duration might be genuinely recent. Zeroing it out creates false confidence in 'this person has no Python' when they just started using it.

Step 3: Endorsement Curve (Logarithmic)

Replace linear endorsement scaling with a logarithmic curve. Going from 0 to 5 endorsements matters much more than going from 45 to 50:

```
| endorsement_factor = min(0.30, log(1 + endorsements) / log(51) × 0.30)
```

0 endorsements → 0.0, 5 endorsements → 0.14, 20 endorsements → 0.22, 50 endorsements → 0.30. Logarithmic scaling prevents a few well-connected candidates from dominating.

Step 4: Assessment Bonus (Platform Verified)

If the candidate has a platform assessment score for this skill, it is the highest-quality signal available — verified externally, not self-reported:

```
assessment_factor = (assessment_score / 100) × 0.25
```

Scores: 0/100 → 0.0, 50/100 → 0.125, 80/100 → 0.20, 100/100 → 0.25

Step 5: Final Trust Score

```
trust_raw = (base_weight × duration_factor) + endorsement_factor +  
assessment_factor  
trust = max(0.05, min(1.0, trust_raw))
```

Minimum floor of 0.05 for any skill on the profile. Maximum cap of 1.0. No negative trust values.

6.3 Computing Category Score

For each of the 6 JD skill categories, compute how well the candidate covers it:

```
category_score = max trust score across all skills in the candidate's  
profile that match this category
```

Use the skill alias map from Module 1 to match variants. If no skill in the category is present:
category_score = 0.

6.4 Final Keyword Trust Score

```
keyword_trust_score = SUM(category_weight × category_score for each of  
6 categories)
```

Category weights from Section 2.2: Embeddings 0.20, Vector DB 0.20, Retrieval 0.20, LLMs/NLP 0.15, Evaluation 0.15, Python/Engineering 0.10.

6.5 Preferred Skills Bonus

Up to +0.08 bonus for preferred skills (capped to prevent preferred skills from overcoming missing hard skills):

```
preferred_bonus = min(0.08, count_of_preferred_skills_matched × 0.02)  
final_skill_score = min(1.0, (0.40 × keyword_trust_score + 0.60 ×  
semantic_score) + preferred_bonus)
```

7. Module 6 — Career Analyzer (Soft Penalty Curves)

7.1 The Four Career Dimensions

Career quality is assessed across four dimensions. All use continuous curves instead of binary thresholds. The four scores are combined into a single career_quality_score.

7.2 Dimension 1: Company Type Quality (Soft Curve)

Step 1: Classify each career role

Company Type	Classification Logic	Base Quality Score
Product Tech Company	Not in consulting list AND industry is Tech/Software/AI/Digital	1.0
Funded Startup	Not consulting, small company size, tech industry	0.95
FinTech / EdTech / HealthTech Product	Not consulting, tech-adjacent industry	0.85
Large Non-Tech Company (IT dept)	Manufacturing, FMCG, Retail with internal IT role	0.65
IT Services / BPO	Not in hard consulting list but IT services model	0.50
Big Consulting (TCS, Infosys etc)	Exact match against consulting firm list	0.30

Step 2: Compute weighted company score with recency bias

```
recency_weight(role) = 2.0 if is_current else 1.5 if role is in last 3
years else 1.0
company_type_score = SUM(base_quality * recency_weight *
duration_months for each role)
/ SUM(recency_weight * duration_months for each
role)
```

This weighted average means recent roles count more than early-career roles. A candidate who spent 5 years at TCS and the last 2 years at a product startup gets a much better score than the raw ratio would suggest.

7.3 Dimension 2: Production Deployment Evidence

Scan all career_history descriptions for production deployment evidence. Keywords are weighted by strength of signal:

Keyword Group	Signal Weight	Example Keywords
Scaling & Infrastructure	High (3 points each)	served N million users, deployed to production, real-time serving, handled X QPS, SLA, latency target

Keyword Group	Signal Weight	Example Keywords
Shipping & Delivery	Medium (2 points each)	shipped, launched, released, went live, rolled out, A/B tested in production, monitored
Engineering Evidence	Low (1 point each)	API, endpoint, model serving, inference pipeline, CI/CD, unit tests, code review, git

```
raw_production_score = SUM(keyword_weight * recency_multiplier for each keyword found)
recency_multiplier = 1.5 if keyword in current or last role else 1.0
production_signal_score = min(1.0, raw_production_score / 15)
```

15 total weighted points = max score. A candidate with 'deployed ML model serving 2M users with 20ms latency SLA, monitored with Grafana' would score near max.

7.4 Dimension 3: Career Trajectory & Seniority

Evaluate the progression of titles over time. Upward trajectory signals growth; flat signals stagnation; current seniority level directly affects fit for a 'Senior AI Engineer' role.

Seniority Map

Title Pattern	Seniority Score
Principal Engineer, Staff Engineer, Distinguished Engineer	1.00
Senior ML/AI/Software Engineer, Tech Lead (IC track)	0.95
ML Engineer, AI Engineer, Data Scientist (4+ years)	0.85
Software Engineer, Data Scientist (2-4 years)	0.70
Junior Engineer, Associate, Analyst	0.50
Manager, Director, VP (unless very recently technical)	0.45 (non-coding leadership concern)
Research Scientist, Researcher (no engineering signals)	0.55 (research without production)

Trajectory Bonus/Penalty

```
title_progression_score = seniority_score_of_current_title
if titles are ascending over time: trajectory_bonus = +0.05
if titles are flat for 7+ years: trajectory_penalty = -0.05
if titles regressed: trajectory_penalty = -0.10
```

7.5 Dimension 4: Industry Relevance


```
industry_score = max(role_industry_score for each career role, weighted  
by recency)
```

Tech/AI industry in current role dominates. Early career in FMCG is irrelevant if current role is in AI.

7.6 Final Career Quality Score

```
career_quality_score = (0.35 × company_type_score  
                        + 0.30 × production_signal_score  
                        + 0.25 × seniority_trajectory_score  
                        + 0.10 × industry_relevance_score)  
                        × disqualifier_soft_penalty
```

The disqualifier_soft_penalty from Module 1 Section 2.2 is applied here to the career score specifically (not the final score). This keeps it proportional and isolated.

8. Module 7 — Behavioral Signal Processor

8.1 Revised Approach: Smooth Multipliers

The blueprint had hard cutoffs for behavioral multipliers (180 days inactive → 0.40×, response rate < 0.05 → 0.50×). The revised version uses smooth exponential decay functions for all behavioral signals. The result is a multiplier that transitions gradually instead of jumping at thresholds.

8.2 Availability Multiplier — Smooth Decay

Component A: Recency of Activity

```
days_inactive = days since last_active_date
recency_multiplier = max(0.20, exp(-days_inactive / 90))
```

At 0 days inactive: 1.00. At 30 days: 0.72. At 90 days: 0.37. At 180 days: 0.14. At 365 days: 0.02. Smooth exponential decay, no cliff.

Component B: Open to Work Flag

```
open_to_work_multiplier = 1.0 if open_to_work_flag = true else 0.45
```

Still a hard signal (it's binary data) but reduced from 0.50 to 0.45 to be slightly less aggressive. The LLM re-ranker can still promote a non-open candidate if they're exceptional.

Component C: Recruiter Response Rate

```
response_multiplier = 0.30 + 0.70 × recruiter_response_rate
```

At response_rate=0.0: 0.30. At 0.5: 0.65. At 1.0: 1.00. Linear interpolation with a 0.30 floor — even someone who never responded before might respond now if the role is good.

Combined Availability Multiplier

```
availability_multiplier = recency_multiplier × open_to_work_multiplier
× response_multiplier
availability_multiplier = max(0.05, min(1.30, availability_multiplier ×
notice_modifier))
```

8.3 Notice Period Modifier

```
notice_modifier = exp(-max(0, notice_period_days - 30) / 80)
```

At 0-30 days: 1.00. At 60 days: 0.69. At 90 days: 0.47. At 120 days: 0.32. Smooth decay from the 30-day target. Long notice periods are a significant practical concern for startups.

8.4 Platform Quality Score

A supplementary additive component (not a multiplier). Added to the base score, not multiplying it.

```
github_contrib = 0 if github_activity_score is None
```

```
        else min(0.12, github_activity_score / 100 × 0.12)
interview_quality = interview_completion_rate × offer_acceptance_rate ×
0.08
        where offer_acceptance_rate defaults to 0.5 if None
profile_quality = max(0, (profile_completeness_score - 60) / 40) × 0.05
platform_quality_score = github_contrib + interview_quality +
profile_quality
```

Platform quality score maximum: ~0.25. It is a soft signal that provides tiebreaking value, not a dominant component.

8.5 Final Behavioral Output

```
behavioral_multiplier = availability_multiplier (range 0.05 to 1.30)
platform_quality_score = (range 0.0 to 0.25)
```

Both outputs are passed separately to the scoring engine. Behavioral multiplier is applied multiplicatively to the base score. Platform quality is added additively. This distinction matters for the score breakdown chart in S8.

9. Module 8 — First-Pass Composite Scorer

9.1 Component Weights

Component	Weight	How It Is Combined
Skill Score (hybrid semantic-keyword)	35%	Additive — direct multiplied weight
Career Quality Score	30%	Additive — direct multiplied weight
Experience Score	10%	Additive — direct multiplied weight
Location Score	10%	Additive — direct multiplied weight
Education Score	5%	Additive — direct multiplied weight
Platform Quality Score	5% (via addition)	Added after weighted sum, not multiplied
Behavioral Multiplier	Applied post-sum	Multiplied against base score — can be 0.05× to 1.30×
Honeypot Penalty	Applied last	Multiplied: 1.0, 0.55, 0.15, or 0.0 based on confidence tier

9.2 Composite Formula

```
base_score = 0.35 × skill_score
            + 0.30 × career_quality_score
            + 0.10 × experience_score
            + 0.10 × location_score
            + 0.05 × education_score
            + platform_quality_score

first_pass_score = base_score × behavioral_multiplier ×
honeypot_penalty
```

9.3 Normalization

```
After computing all 100K first_pass_scores:
normalised_score = first_pass_score / max(all first_pass_scores)
```

Sort all 100K candidates by `normalised_score` descending. Select top 200-300 for LLM re-ranking phase. This threshold is configurable — higher threshold = less LLM cost but may miss good candidates ranked 101-200 by the algorithm.

9.4 Shortlist Threshold

Threshold Choice	LLM API Calls (batch 5)	Risk	Recommendation
------------------	-------------------------	------	----------------

Threshold Choice	LLM API Calls (batch 5)	Risk	Recommendation
Top 100 only	20 calls	May miss good candidates at ranks 101-200	Too risky — good candidates may be close to the cutoff
Top 200	40 calls	Low risk — reasonable coverage	RECOMMENDED for most cases
Top 300	60 calls	Very low risk — strong coverage	Use if LLM cost is not a concern

10. Module 9 — LLM Re-Ranker (KEY NEW MODULE)

WHY THIS EXISTS: The scoring formula is deterministic and rule-based. It cannot understand that 'built a system to find relevant job listings for users based on what they've looked at before' is equivalent to 'experience with retrieval and ranking systems'. The LLM can. This module is the semantic understanding layer that closes the gap between what the JD says and what candidates actually did.

10.1 Overview of Two-Phase Process

Phase 1 (Algorithmic) produces top 200-300 candidates from 100K. Phase 2 (LLM) re-evaluates these 200-300 with deep reading and produces a nuanced score for each. The final score blends both phases.

10.2 Candidate Summary Builder

Before calling the LLM, build a compact but information-rich summary for each candidate. This summary is what the LLM reads. It must contain everything relevant without exceeding token limits.

Section	Content Included	Why
Identity Line	'<title>, <YoE> years, <location>, <company_type tag>'	Orients the LLM immediately
Career Highlights	Latest 3 roles: title at company (duration): first 150 chars of description. Latest first.	Most recent experience is most relevant — older roles add context only
Claimed Skills	Top 8 skills sorted by trust score (highest trust first, not alphabetically)	Shows the LLM what's credibly claimed — pre-filtered by trust
Availability Summary	'Open to work: Y/N. Last active: X days ago. Notice: Y days. Response rate: Z%'	Lets LLM factor in availability naturally
Red Flags (if any)	'Note: all career at consulting firms' or 'Note: only recent AI experience (<12 months)'	Pre-flag known concerns so LLM can weigh them explicitly

Total summary length: 400-600 tokens per candidate. This keeps batch calls manageable at 5 candidates per batch (~3000 tokens total per call).

10.3 JD Context Passed to LLM

Send a structured, compact JD context with every batch call. This is NOT the full job description — it is the parsed requirements object from Module 1:

- Hard requirements: one-line description of each required skill group
- Key disqualifiers: the 5 disqualifying patterns in plain English
- Ideal profile description: '5-9 years, product company experience, production ML deployment, retrieval/ranking/search systems experience'

- Scoring guidance: 'Score 0-100. 85+ = exceptional fit. 60-84 = good fit. 40-59 = marginal. <40 = poor fit.'

10.4 Prompt Structure

System Prompt

'You are an expert senior technical recruiter specializing in AI/ML engineering roles. You evaluate candidates rigorously and fairly. You can identify genuine AI/ML engineering expertise even when candidates describe their work in plain language without jargon. You output only valid JSON with no additional text.'

User Prompt — Batch of 5 Candidates

Structure the user prompt as:

- Section 1: JD context (compact version, ~200 tokens)
- Section 2: 'Evaluate these 5 candidates against the role. For each, return: candidate_id, score (0-100), reason (one sentence, specific to their profile), and any major concern.'
- Section 3: Candidate 1 summary, Candidate 2 summary, ... Candidate 5 summary
- Section 4: 'Respond ONLY with a JSON array:
[{"id":"CAND_XXXXXXX","score":85,"reason":"...", "concern":"none" or "specific concern"}, ...]'

Why Batch Size 5?

- Enables within-batch relative comparison — LLM naturally compares candidates against each other within a batch, improving relative ranking
- Keeps per-call token count manageable (~3000 tokens)
- Allows LLM to allocate different scores across the batch (prevents all-80 syndrome)
- 5 candidates × 40 batches = 200 candidates covered in 40 API calls

10.5 Response Parsing

Parse the JSON response to extract per-candidate score and reason:

- Extract the JSON array from the response (strip any markdown backticks if model wraps output)
- For each item: read id, score, reason, concern
- Validate: score must be 0-100 integer or float, id must be CAND_XXXXXXX format
- Store: llm_score[candidate_id] = score, llm_reason[candidate_id] = reason

10.6 Error Handling & Fallback

Error Case	Detection	Handling
API timeout (>15s)	Request exceeds timeout	Retry once with 5s wait. If still fails: use algorithmic score only (llm_weight = 0 for this batch)
Malformed JSON response	json.loads throws exception	Retry once with temperature=0. If still fails: use algorithmic score for affected candidates

Error Case	Detection	Handling
Missing candidate_id in response	id not found in response batch	Log warning. Use algorithmic score for that candidate. Do not fail entire batch.
Score out of range	score < 0 or score > 100	Clamp to [0, 100]. Do not discard the response.
Rate limit hit	429 response code	Exponential backoff: wait $2^{\text{retry_count}}$ seconds. Max 3 retries.
LLM completely unavailable	All retries fail	Fall back to 100% algorithmic ranking. Log the fallback for transparency.

RESILIENCE RULE: The LLM re-ranker must NEVER block or crash the ranking pipeline. Every error state has a defined fallback to the algorithmic score. A submission with pure algorithmic ranking is better than no submission.

10.7 Score Blending

After all LLM scores are collected, blend with the first-pass algorithmic scores:

```
normalized_algo_score = first_pass_score / max(all first_pass_scores in
shortlist)
normalized_llm_score = llm_score / 100
blended_score = (1 - llm_weight) × normalized_algo_score + llm_weight ×
normalized_llm_score
default llm_weight = 0.60
```

If LLM call failed for a candidate: llm_weight = 0 for that candidate (pure algorithmic). This ensures failed LLM calls degrade gracefully.

10.8 Final Rank Determination

After blending, candidates NOT in the shortlist (ranks 201-100K from the first pass) retain their first_pass_score with a small ceiling cap:

```
score_for_non_shortlisted = first_pass_score × 0.85
```

The 0.85 cap ensures that a very good non-shortlisted candidate cannot beat a LLM-evaluated shortlisted candidate, even if their algorithmic score was slightly higher. This maintains the integrity of the LLM re-ranking phase.

Sort all candidates by their final score (blended for shortlisted, capped for non-shortlisted). Select top 100. These are the final submission candidates.

10.9 Model Choice

Model	Speed	Cost (200 candidates)	Quality	Notes
Gemini 1.5 Flash	Very fast (~1s/batch)	<\$0.01	Good	Best for hackathon — near-zero cost, fast
Gemini 1.0 Pro	Fast (~2s/batch)	~\$0.05	Very good	Good balance

Model	Speed	Cost (200 candidates)	Quality	Notes
GPT-4o-mini	Fast (~2s/batch)	~\$0.02	Very good	Excellent JSON reliability
GPT-4o	Slower (~4s/batch)	~\$0.20	Best	Overkill for this task — cost not justified
Anthropic Claude Haiku	Fast (~1.5s/batch)	~\$0.01	Good	Alternative if others unavailable

RECOMMENDATION: Use Gemini 1.5 Flash as primary model. Set temperature=0.1 for consistency. Configure GPT-4o-mini as fallback if Gemini is unavailable. Both produce reliable JSON output when prompted correctly.

11. Module 10 — Reasoning Generator

11.1 LLM-Powered with Grounding

In the revised system, reasoning is generated by the same LLM used for re-ranking — the re-ranker already produces a one-sentence reason for each candidate as part of its response. The reasoning generator enriches this reason with structured data from the candidate's profile.

11.2 Grounding Process

The LLM reason (e.g., 'Built production retrieval system at a fintech startup') must be verified and enriched with exact data points from the profile before being stored in the output.

LLM Reason Contains	Grounding Lookup	Final Enriched Output
Title reference	Verify against <code>profile.current_title</code> . Correct if wrong.	Use exact profile title, not LLM's paraphrase
Company reference	Verify against <code>career_history</code> . Identify which company.	Replace vague references with actual company name
Skill reference	Verify the claimed skill exists in profile skills AND has trust > 0.3	Remove or flag skills the LLM mentioned that have very low trust
Duration reference	Cross-check any 'N years' claim against computed YoE	Use exact <code>years_of_experience</code> value
Location/Availability	Append verified location, notice days, response rate to end of reason	Always grounded — never hallucinated location data

11.3 Final Reasoning Format

```
Sentence 1: [LLM-generated, grounded] – what makes this candidate fit the role
Sentence 2: [Data-appended] – location + notice period + availability signal
```

If LLM reason is already 2 sentences: use both directly (after grounding). If LLM reason is very short (<50 chars): supplement with a second sentence built from profile data. Never exceed 2 sentences.

11.4 Concerns Handling

If the LLM's 'concern' field for this candidate is not 'none', append it to the reasoning:

```
If concern exists: Sentence 2 = '[availability/location data]. Note: [concern].'
```

Example: 'ML Eng, 6.8yrs; built semantic search at Meesho (FAISS, NLP). Bangalore, 30-day notice. Note: last active 92 days ago — confirm availability before outreach.'

Being honest about concerns in reasoning is a positive signal to judges — it shows the system doesn't blindly promote candidates and understands recruiter needs.

12. Module 11 — Output Pipeline & Validator

12.1 Score Finalization

After blending and final ranking, apply two final operations before writing output:

- Tie-breaking: if two candidates have the same blended_score (to 6 decimal places), sort by candidate_id ascending. This is the submission spec requirement.
- Score scaling: normalize the final 100 scores so rank-1 candidate = 1.000 and scores are non-increasing. Formula: scaled_score = blended_score / max(top100_blended_scores). This ensures the output always has a 1.000 at rank 1.

12.2 Output Schema

Column	Format	Validation Rule
candidate_id	String: 'CAND_XXXXXXX' (7 digit zero-padded)	Must exist in input candidates.jsonl. No invented IDs.
rank	Integer 1-100	Each value 1-100 appears exactly once. No gaps. No duplicates.
score	Float, 6 decimal places, 0-1	Non-increasing: score[rank=n] >= score[rank=n+1] for all n. Ties allowed.
reasoning	String, max 500 characters, no newlines inside cell	1-2 sentences. No quotes around the whole text. CSV-escaped if contains commas.

12.3 Pre-Submission Validation Checklist

Run these checks before writing the file. Reject and log any violations:

- Row count = 100 (exactly, not 99 or 101)
- All ranks 1-100 present, each exactly once
- All candidate_ids are valid format (CAND_XXXXXXX) and exist in the input
- No duplicate candidate_ids
- All scores in [0, 1]
- Scores are non-increasing from rank 1 to 100
- All reasoning fields are non-empty, under 500 characters, no internal newlines
- File is UTF-8 encoded
- Honeypot candidates in top 100: count must be < 10 (target: 0)

13. API Endpoint Specifications — Frontend-Backend Contract

Every API endpoint, what it receives, what it returns, and which frontend module it feeds. These specifications must be implemented exactly — the frontend was designed to consume these response shapes.

GET /api/status

Direction	Details
Purpose	Check system readiness: are embeddings loaded? Is JD parsed? Is model warm?
Returns	{ status: 'ready' 'loading' 'error', message: string, details: { embeddings_loaded: bool, jd_parsed: bool, candidates_loaded: int } }
Feeds	S1-C System Status Badge, S4-E Proceed Button enabled state

GET /api/jd

Direction	Details
Purpose	Return the fully parsed JD requirements for display in S3
Returns	{ hard_skills: [{name, weight, keywords, semantic_equivalents}], preferred_skills: [{name, weight}], disqualifiers: [{name, description, penalty_function_description}], location_map: {city: score}, experience_curve: {ideal_min, ideal_max, peak_min, peak_max}, jd_queries: ['query text 1', 'query text 2', 'query text 3'] }
Feeds	S3-A (JD highlighting), S3-B (extracted requirements), S3-C (location matrix), S3-D (experience range), S3-E (firm list)

POST /api/upload

Direction	Details
Purpose	Accept uploaded candidate file. Validate schema. Return candidate count and preview.
Request	multipart/form-data: file (.json or .jsonl), max 100 candidates for demo
Returns	{ success: bool, count: int, skipped: int, preview: [{candidate_id, current_title, location, years_of_experience}] (first 3 candidates), errors: [string] }
Feeds	S4-C Validation Status, S4-D Preview Strip, S4-E enable/disable Proceed button

POST /api/rank

Direction	Details
Purpose	Run the full ranking pipeline on loaded candidates. Returns streaming progress events + final results.
Request	{ candidates: [candidate object], weights?: {skill, career, experience, location, education},

Direction	Details
	thresholds?: {inactive_days, response_rate, notice_days}, llm_enabled: bool }
Progress Events (Server-Sent Events)	{ stage: 1-8, stage_name: string, progress: 0-100, candidates_processed: int, message: string }
Final Response	{ results: [{candidate_id, rank, score, reasoning, features: {skill_score, career_score, experience_score, location_score, education_score, behavioral_multiplier, platform_quality_score, honeypot_confidence, llm_score, llm_reason, llm_concern, disqualifier_flags: [string]}}, ...], runtime_seconds, llm_enabled: bool, llm_model_used: string }
Feeds	S6 all modules (progress), S7 full results table, S8 complete detail view, S9 analytics, S10 export, S11 LLM inspector

POST /api/validate

Direction	Details
Purpose	Validate the current ranking results against submission spec rules.
Request	{ results: [same format as /api/rank response results array] }
Returns	{ valid: bool, checks: [{name: string, passed: bool, message: string}] }
Feeds	S10-B Submission Format Validator

POST /api/export

Direction	Details
Purpose	Generate and stream the submission CSV file.
Request	{ results: [same format], filename: string }
Returns	CSV file stream (Content-Type: text/csv, Content-Disposition: attachment)
Feeds	S10-C Download CSV Button

GET /api/candidate/:candidate_id

Direction	Details
Purpose	Return full profile + computed features for a specific candidate (for S8 detail view).
Returns	{ profile: {all candidate profile fields}, features: {all computed scores}, reasoning: string, rank: int, score: float, llm_details: {score, reason, concern, model_used} }
Feeds	All S8 sub-modules (profile header, score breakdown, skills table, career timeline, behavioral grid, honeypot results, LLM panel)

14. Pre-Computation Strategy

Task	Run When	Output	Purpose
Embed all 100K candidate texts using sentence-transformers	Once offline. ~25-55 min on CPU.	candidate_embeddings.npy (shape 100K×384, ~150MB)	Eliminates embedding overhead from online ranking. Load in seconds at runtime.
Compute JD embeddings (3 queries)	Once, whenever JD changes.	jd_embeddings.npy (shape 3×384, tiny)	Pre-computed for instant cosine similarity at runtime.
Pre-compute non-embedding features for all 100K	Once offline. ~5-10 min.	features_cache.pkl (~100MB). Contains: honeypot_confidence, company_type_score, production_signal_score, seniority_score, experience_score, location_score, education_score, behavioral_multiplier per candidate.	Reduces online ranking to: load files + vector multiply + sort. Target: under 90 seconds for full 100K.
Build candidate summary texts for LLM	At start of online ranking, for shortlist only.	In-memory dict of 200-300 summary strings.	No need to pre-compute — only 200-300 summaries needed and they're fast to build.

TOTAL ONLINE RUNTIME ESTIMATE: Load embeddings (5s) + Load features cache (3s) + Compute cosine similarities 100K×3 (10s) + First-pass scoring (15s) + Sort and shortlist (2s) + LLM re-ranking 40 batches (60s) + Final blend and rank (3s) + Generate output (5s) = ~103 seconds. Well within 5-minute constraint.

15. Fallback & Recovery Logic

The system must submit successfully even when individual components fail. Here is the fallback hierarchy for each critical failure point:

Failure Point	Detection	Fallback	Quality Impact
candidate_embeddings.npy not found	FileNotFoundError on load	Skip semantic score. Use keyword_trust_score as full skill_score (remove blending). Log prominently.	Moderate — lose semantic advantage but still functional
features_cache.pkl not found	FileNotFoundError on load	Recompute features online. Will be slower (may take 3-4 minutes). Still within 5-min budget with small sample.	No quality impact. Time budget consumed.
LLM API completely unavailable	All retries fail across all batches	Set llm_weight = 0. Final ranking = pure algorithmic (first_pass_score). Submission still valid.	Moderate — lose LLM judgment but system still produces valid output
LLM returns invalid JSON	json.loads fails after retry	Use algorithmic score for affected batch. llm_score[affected candidates] = first_pass_score × 100 (treated as neutral)	Minimal — only 5 candidates per batch affected
Candidate count < 100	Only 80 valid candidates after honeypot filtering	Include all valid candidates. submission has < 100 rows. Validate: submission_spec may allow <100 if fewer input candidates.	Depends on sample — check submission spec
Memory overflow during feature computation	MemoryError	Reduce batch size. Process candidates in chunks of 10K instead of all at once.	No quality impact

END OF BACKEND LOGIC BIBLE

11 Modules | 10-Stage Pipeline | Semantic-First | LLM Re-Ranking | Soft Penalty Curves